

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	Cao Trần Bá Đạt
Student ID:	23127168
Class / Cohort:	23KTPM1
Assignment ID (e.g., HW#00, HW#02):	HW#03
Assignment date:	03/08/2026
AI tool(s) used:	Gemini
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

Name of the AI tool	Date and time	Your prompt	The AI output
Claude	23:47 01/08/2026	This is the README file describing the Eshop system and the JSX file for Login.	Looking at the code against the FR-02 spec, there's a bigger problem than selector strategy: the current markup makes reliable selectors nearly impossible, and there are spec violations you'll want to log as defects

Name of the AI tool	Date and time	Your prompt	The AI output
		Imagine you are a professional QA/QC professional, and I want to write test automation for the FR-02 feature. Please suggest the most optimal list of selectors (CSS/XPath/Role), minimizing the use of fragile selectors. Attached: Login.jsx, README.md	alongside the automation. ### Defects found (log these, they affect what "correct" selectors even look like): 1. Heading says "Đăng Ký" (Register) on the Login page (Content bug — confusing test oracle) - 2. Password <code><input></code> uses <code>type="text"</code>, not <code>type="password"</code> (FR-22: "Trường Mật khẩu phải dùng <code>type="password"</code>") - 3. Email <code><input></code> uses <code>type="text"</code>, no <code>type="email"</code> (FR-02: "Trường email phải dùng <code>type="email"</code>") - 4. <code><label></code> and <code><input></code> are siblings with no <code>htmlFor/id</code> link (Accessibility + breaks <code>getByLabel()</code>) - 5. No <code>id</code>, <code>name</code>, or <code>data-testid</code> on either input (Forces fragile structural selectors) - 6. Submit button text is "Sign In" (English) (FR-21: "Nhất quán ngôn ngữ... dùng tiếng Việt"). Because of #4 and #5, you genuinely can't build a robust selector for the inputs today — anything you write will be a workaround, not a "good" selector. I'd flag this to the dev team as a testability blocker before writing final automation. ### Selector recommendations: Email input: <code>page.getByTestId('login-email-input')</code> (Best available today: <code>page.locator('form > div').nth(0).locator('input')</code>) - Password input: <code>page.getByTestId('login-password-input')</code> (Best available today: <code>page.locator('form > div').nth(1).locator('input')</code>) - Submit button: <code>page.getByRole('button', { name: 'Đăng nhập' })</code> - "Quên mật khẩu?" link: <code>page.getByRole('link', { name: 'Quên mật khẩu?' })</code> - "Đăng ký ngay" link: <code>page.getByRole('link', { name: 'Đăng ký ngay' })</code> - Error message: <code>page.getByTestId('login-error')</code>. ### Why role-based selectors for the inputs won't work yet: <code>getByRole('textbox', { name: 'Email' })</code> / <code>getByLabel('Mật khẩu')</code> are what you'd normally reach for first — but the accessible name comes from <code><label for></code> → <code><input id></code> association, which doesn't exist here. Until fix #4 lands, any selector targeting the inputs is inherently positional or type-based. ### FR-02 test scenarios: Valid login, Invalid password, 3rd consecutive failed attempt (lockout), Lockout expiry (30s), Email field HTML5 validation.

Name of the AI tool	Date and time	Your prompt	The AI output
Claude	23:55 01/08/2026	Create a test data file in .json or .csv format containing at least 12 test cases for this feature (including Positive, Negative, and Edge cases). Export the sample data as a table/series.	15 test cases covering FR-02 (2 Positive, 8 Negative, 5 Edge/Security), including the full 3-strike lockout sequence, the 30s expiry boundary, HTML5 validation gaps, an SQLi probe, and a couple of undefined-by-spec cases (email case sensitivity, password whitespace) worth flagging as clarification items rather than pass/fail. A few call out known defects from the earlier review (TC-LOGIN-11 will fail today because the email field isn't <code>type="email"</code>) — useful for tying automation results back to the bug list. Attach: FR-02_Login_Lockout_TestData.csv
Claude	23:57 01/08/2026	Based on the Locators and Test Data above, write a Playwright script (Typescript) that applies data-driven testing. You are required to use at least three different types of Assertion (e.g., <code>toBeVisible</code> , <code>toHaveText</code> , <code>toHaveURL</code>).	Two files: <code>login.spec.ts</code> (the suite) and <code>loginTestData.json</code> (the data it loops over — drop both in the same folder since the spec imports the JSON directly). Structure: Data-driven block (loops <code>loginTestData.json</code> - 5 cases), Serial lockout sequence (3 failed attempts → locked → correct password still rejected → succeeds after 30s wait), Required-field validation (checks native HTML5 <code>validity.valid</code>), Known-defect block (asserts <code>type="password" / type="email"</code> , marked <code>test.fail()</code>). Assertion types used: <code>toHaveURL</code> , <code>toBeVisible/not.toBeVisible</code> , <code>toHaveText</code> , <code>toHaveAttribute</code> , <code>expect().toBe()</code> . One caveat worth flagging: locators for inputs are still positional ones (<code>form > div >> nth(n) >> input</code>) due to missing <code>data-testid</code> . Attach: Login.spec.ts , LoginTestData.json
Claude	23:59 01/08/2026	Add configuration logic to run this script on 3 browsers (Chromium, Firefox, WebKit) and integrate HTML report output (Playwright HTML Reporter/Allure) that displays 'Run by: {23127168}'.	Five files now, all meant to sit in the same project root: <code>playwright.config.ts</code> (3 projects: chromium, firefox, webkit + reporters), <code>global-setup.ts</code> (writes <code>allure-results/environment.properties</code> with <code>Run by=23127168</code>), <code>login.spec.ts</code> (added <code>test.beforeEach</code> annotation for HTML report), <code>package.json</code> (scripts to run per-browser or all three, plus generate/open reports). How to run: 1. <code>npm install</code> - 2. <code>npx playwright install</code> - 3. <code>npm test</code> - 4. <code>npm run report:html</code> - 5. <code>npm run report:allure:generate && npm run report:allure:open</code> . Attach: global-setup.ts ,

Name of the AI tool	Date and time	Your prompt	The AI output
			login.spec.ts , loginTestData , package.json , playwright.config.ts
Claude	19:40 02/08/2026	This is the JSX file for the Checkout function. Please suggest a list of the most optimal selectors (CSS/XPath/Role), minimizing the use of error-prone selectors for the FR-09 function requirement in the README file, similar to what you did above. Attached: Checkout.jsx	Reviewed against FR-09 (and direct dependency FR-08). Zero <code>data-testid</code>, <code>id</code>, or <code>name</code> attributes anywhere in this component, plus serious business-logic defects. ### 🐛 Defect to flag: 1. <code>editableTotal</code> is a free-text <code><input type="number"></code> the user can directly overwrite (FR-08 violation) - 2. Client-controlled <code>editableTotal</code> is sent as <code>total_amount</code> to <code>/api/apply-coupon</code> (FR-09 C3 integrity issue). ### 🎯 Selector table: Coupon code input: <code>page.getByPlaceholder('Nhập mã giảm giá...')</code> - Apply coupon button: <code>page.getByRole('button', { name: 'Áp dụng' })</code> - Coupon error message: <code>page.locator('.text-red-600.text-sm')</code> - Coupon success message: <code>page.getByText(/✅/)</code> - Discount amount: <code>page.locator('div.text-green-700 p', { hasText: 'Tiết kiệm' }).locator('strong')</code> - Final amount: filtered on <code>'Thành tiền'</code> - Editable total input: <code>page.getByRole('spinbutton')</code> - Bottom total display: <code>page.getByText(/Tổng thanh toán:/)</code> - Checkout button: <code>page.getByRole('button', { name: 'Xác Nhận Thanh Toán' })</code> - Success heading: <code>page.getByRole('heading', { name: 'Thanh toán thành công!' })</code>. ### 💡 Recurring theme: Dynamic-state buttons change accessible names during loading, creating flaky role-based selectors. Recommend adding <code>data-testid</code>.
Claude	20:15 02/08/2026	Create a test dataset in .json or .csv format containing at least 12 test cases for this feature, similar to the method described above.	15 test cases covering FR-09 (3 Positive, 5 Negative, 7 Edge/Security), each mapped explicitly to one of the five conditions (C1–C5) so it's clear which rule a failure isolates: C1 (code exists) — TC-04 - C2 (not expired) — TC-05 - C3 (threshold met, including the $\geq$ boundary) — TC-06, TC-09, TC-10 - C4 (logged in) — TC-07 - C5 (usage cap, single-use and multi-use) — TC-08, TC-15 - Plus normalization checks (TC-11 lowercase, TC-12 whitespace) and one flagged Critical security case (TC-COUPON-13) that specifically exploits the editable-total defect from the earlier selector review — it applies a coupon using a fabricated total, then checks whether the backend actually recomputes the real amount at

Name of the AI tool	Date and time	Your prompt	The AI output
			checkout rather than trusting the client, which is the real point of failure here rather than the coupon logic itself. Attach: FR-09_Coupon_TestData.csv
Claude	20:36 02/08/2026	These are the playwright.config and global-setup files. Use these files, along with the Locators and Test Data above, to write a script for this feature similar to the FR-02 requirements. Note that you should display 'Run by: {23127168}'. Attach: global-setup.ts , playwright.config.ts	Three files (placed alongside existing setup): couponTestData.json (6-case data-driven subset), checkout.spec.ts (LoginPage + CheckoutPage POMs), and updated global-setup.ts (stamps Features=FR-02 Login & Account Lockout, FR-09 Coupon). Structure: Data-driven block (loops couponTestData.json - 6 cases), Serial block for stateful C5 usage-cap cases (SAVE10 single-use, VIP100 multi-use), Boundary block for C3 >= threshold, Security block (TC-COUPON-13 tampering editableTotal to document FR-08 gap), Guard block (TC-COUPON-14 disabled button state). Annotations & Reporters: test.beforeEach pushes "Run by: 23127168" chip for HTML report; global-setup.ts writes Allure environment.properties . Assertion types: toBeVisible/not.toBeVisible , toContainText , toBeDisabled . Note: CheckoutPage.goto() assumes seeded cart state (FR-06/07 dependency); seedCart() helper recommended for full self-containment. Attach: couponTestData.json , checkout.spec.ts , global-setup.ts .
Claude	23:58 02/08/2026	This is the JSX file for the admin function. Please suggest a list of the most optimal selectors (CSS/XPath/Role), minimizing the use of error-prone selectors for the FR-17 function requirement in the README file, similar to what you did above. Attach: App.js	Reviewed the Coupon CRUD section (FR-17, <code>activeTab === "coupons"</code> , lines 612–773). Same selector-fragility root cause as before (no id/name/data-testid anywhere), plus a couple of spec-vs-implementation gaps worth logging first. ### 🐞 Defects to flag: 1. Every form field uses only a placeholder — there is no <code><label></code> element anywhere in this form (FR-22 violation: "Tất cả trường bắt buộc phải có ký hiệu * bên cạnh nhãn") - 2. <code>min_order_amount</code> input has no required attribute (FR-17 explicitly lists <code>min_order_amount >= 0</code> as a required field) - 3. <code>discount_value</code> has required but no <code>min="0"/positive-number</code> constraint — a negative or zero value is submittable client-side (FR-17: " <code>discount_value (dương)</code> ") - 4. The <code>discount_value</code> placeholder text changes depending on <code>couponForm.type</code> ("Giá trị % (VD: 10)" ↔ "Số tiền (VD: 50000)"), which directly breaks any <code>getByPlaceholder()</code> selector assuming a fixed string.

Name of the AI tool	Date and time	Your prompt	The AI output
			### 🧐 Selector table: "Mã Giảm Giá" sidebar tab: <code>page.getByRole('listitem').filter({ hasText: 'Mã Giảm Giá' })</code> - Coupon code input: <code>page.getByPlaceholder('Mã coupon (VD: SAVE10)')</code> - Type dropdown: <code>page.locator('form select')</code> or <code>page.getByRole('combobox')</code> - Discount value input: <code>page.getByPlaceholder('/Giá trị %&#124;Số tiền/')</code> (🔴 High fragility due to dynamic placeholder) - Min order amount input: <code>page.getByPlaceholder('Đơn tối thiểu (đ)')</code> - Expiry date input: <code>page.locator('input[type="date"]')</code> - Max uses/user input: <code>page.getByPlaceholder('Số lần dùng tối đa/người')</code> - Submit ("Tạo mã"): <code>page.getByRole('button', { name: 'Tạo mã' })</code> (🟢 Low fragility) - A specific coupon's table row: <code>page.getByRole('row', { name: /SAVE10/ })</code> - Delete button for a specific coupon: <code>page.getByRole('row', { name: /SAVE10/ }).getByRole('button', { name: 'Xóa' })</code> - Table column headers: <code>page.getByRole('columnheader', { name: 'Đơn tối thiểu' })</code>. ### 💡 Summary: Every input field is placeholder-only with no stable hook — adding <code>data-testid</code> to inputs would upgrade nearly every 🟡 row above to 🟢 in one pass.
Claude	00:03 03/08/2026	Create a test dataset in .json or .csv format containing at least 12 test cases for this feature, similar to the method described above	15 test cases for FR-17 (3 Positive, 5 Negative, 7 Edge), same defect-aware approach as the last two datasets: Boundary coverage on numeric constraints (<code>min_order_amount >= 0</code> via TC-10 at 0, <code>max_uses_per_user >= 1</code> via TC-12 at 0) - Validation-gap cases (TC-08, TC-09, TC-11) deliberately submit values the client won't stop (zero/negative <code>discount_value</code>, negative <code>min_order_amount</code>) to assert on backend rejection - TC-14 is an API-level case checking FR-12/SEC-03 role enforcement directly - TC-13 (Low priority) documents upper bound behavior on percent discounts. Attach: FR-17_CouponAdmin_TestData.csv
Claude	00:19 03/08/2026	This is the script file that runs for the checkout part of the	Two files added (following checkout.spec.ts pattern): <code>adminCouponTestData.json</code> (7-case data-driven subset: 2 positive creates, <code>min_order_amount = 0</code>

Name of the AI tool	Date and time	Your prompt	The AI output
		FR-09 request; I've added a section about resetting the database. Based on the Locators and Test Data above and the sample script above, write a Playwright (Typescript) script that applies data-driven testing. You need to use at least three different Assertions (e.g., toBeVisible, toHaveText, toHaveURL). Note that test cases capable of running automation must be included in the script, and the script should also reset the database if necessary. Attach: checkout.spec.ts	boundary, and 4 spec-correct negative cases) and coupon-admin.spec.ts (Admin CRUD test suite). Suite Structure: Data-driven block (loops adminCouponTestData.json), Duplicate-code isolation (row-count comparison before/after submit), HTML5 validation & boundary checks (validity.valid assertion for empty fields & max_uses_per_user = 0), Stateful Delete flow (creates then deletes, asserts row disappearance), Input normalization test (lowercase input flashsale transforms to FLASHSALE), and direct API-level RBAC test (uses request fixture with regular user JWT to hit /api/admin/coupons , asserting 401/403 status). Technical Note: Unlike customer-facing inline errors, Admin form errors trigger via native alert() , handled using page.waitForEvent('dialog') + dialog.accept() via submitExpectingError() helper. Assertions used: toBeVisible/not.toBeVisible , toContainText , toHaveCount , toHaveValue , plus expect().toBe().toContain() . Attach: adminCouponTestData.json , coupon-admin.spec.ts . Attach: adminCouponTestData.json , coupon-admin.spec.ts

4. Conclusion — When should AI be used (or not)?

In conclusion, Claude demonstrated strong capabilities in generating Playwright test scripts for HW04, successfully setting up robust baseline frameworks, selecting reliable fallback locators, and covering primary UI workflows across multiple features like login, checkout, and coupon management. However, the initial AI-generated implementations revealed critical structural and logical flaws when evaluated for long-term scalability and test reliability. Across FR-01, FR-09, and FR-17, the AI consistently omitted key edge cases from requirement specifications, struggled with stateful backend constraints—such as failing to reset the database, which caused flaky failures in sequential runs—and introduced unsuitable configurations like parallel execution that triggered race conditions on shared state. Additionally, its rigid assertion logic mishandled dynamic UI formatting like VND currency separators, while neglecting key data attributes in generated loops. Through manual refactoring—implementing automated database re-seeding hooks, switching to sequential execution (test.describe.serial), adopting flexible regex assertions, and expanding test coverage—we transformed the AI's boilerplate code into a highly reliable, isolated, and resilient test suite. Ultimately, while the AI provided a valuable starting foundation, human QA oversight remains essential to ensure complete coverage, state isolation, and accurate assertions in real-world automation testing.

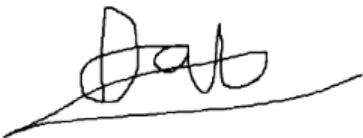
5. Mandatory Disclosure

The test cases, test scripts were initially generated by Claude; I reviewed and modified the test data json; the automation script review, the bug report and the test summary was written entirely by me. The HTML reports correctly contain "Run by: {StudentID}". The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

Signature

Student name (printed):	CAO TRẦN BÁ ĐẠT
Student ID:	23127168
Class / Cohort:	23KTPM1
Course:	CS423 / CSC13003 – Software Testing
Instructor:	LÂM QUANG VŨ
Date:	03/08/2026

Signature:



Cao Trần Bá Đạt

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.